Data Structures

BS (CS) _Fall_2025

Lab_03 Task

Learning Objectives:

1. 2D and 3D arrays

Task 0: Templates with Arrays (Insert, Delete, Display)

Problem Statement:

Create a template class Array3D<T> that stores elements in a 3D dynamic array.
The class should support basic insert, delete, and display operations.

Features to Implement:

Constructor

- Takes 3 dimensions (x, y, z) as input.
- Allocates a dynamic 3D array of that size.
- Initializes all values to 0 (or T{} default constructor).

insert(int x, int y, int z, T value)

- Insert value at position [x][y][z].
- If indices are out of range, show an error message.

deleteAt(int x, int y, int z)

- Delete element at [x][y][z] by resetting it back to 0 (or T{}).
- If indices are out of range, show an error message.

printArray()

- Print the entire 3D array in a readable format.

Google Test Cases (GTest)

1. Basic Array Tests (T = int)
    - Create Array3D<int> arr(2, 2, 2).
    - Verify all elements initialized to 0.
    - Insert 42 at (0,1,1) → element at (0,1,1) should be 42.
    - Insert value at out-of-range index (e.g., (3,0,0)) → should show error.
    - Delete element at (0,1,1) → element should reset to 0.
    - Delete element at out-of-range index (2,2,2) → should show error.
    - Print array after some insertions → output should contain inserted values.
2. Different Data Type Tests
    - Array3D<double>
    - Create Array3D<double> arr(2,2,1).
    - Insert 3.14 at (1,0,0).
    - Delete (1,0,0) → value resets to 0.0.

- Array3D<char>
- Create Array3D<char> arr(2,2,2).
- Insert 'A' at (0,0,1).
- Delete (0,0,1) → value resets to '\0'.

Task 1: Image Processing (Matrix Operations)

Problem Statement

An image can be represented as a 2D matrix of pixel intensities (values between 0 and 255).

For example, a 3×3 grayscale image:

```
[100 150 200]
[  50  75 125]
[   0   0 255]
```

Each entry is a pixel brightness:

- 0 = black
- 255 = white
- values in between = shades of gray.

We want to implement a template class ImageMatrix<T> (using dynamic 2D arrays) that can perform basic image transformation

Features to Implement

transpose()

- Flips the image along the main diagonal.
- Rows become columns, columns become rows.

add(ImageMatrix other)

- Adds two images of the same size pixel by pixel.
- This is like blending two frames.

multiply(ImageMatrix other)

- Perform matrix multiplication of two matrices.
- In image processing, this can simulate applying a filter (like sharpening or blurring).
- (AB)[i][j] = Σ (A[i][k] * B[k][j])

rowAverage(row)

- Compute the average brightness of a given row.
- Useful for analyzing brightness levels across the image.

printImage()

- Display the image in a grid form.

Example Scenario

Original Image (3×3):

```
[100 150 200]
[ 50  75 125]
[  0   0 255]
```

- **Transpose :-** flips along diagonal.
- **Add with another image** :- blends two frames.
- **Multiply with filter matrix** :- sharpens or blurs.

GTEST for Image Processing:

```
// Test transpose
TEST(ImageMatrixTest, TransposeWorks) {
    ImageMatrix<int> img(3, 3);
    int values[3][3] = {
        {100, 150, 200},
        { 50,  75, 125},
        {  0,   0, 255}
    };
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            img.setValue(i, j, values[i][j]);
        }
    }

    img.transpose();

    EXPECT_EQ(img.getValue(0,1), 50);
    EXPECT_EQ(img.getValue(1,0), 150);
    EXPECT_EQ(img.getValue(2,0), 200);
    EXPECT_EQ(img.getValue(0,2), 0);
}

// Test add
```

```
TEST(ImageMatrixTest, AddTwoImages) {
    ImageMatrix<int> img1(3,3), img2(3,3);
    int values[3][3] = {
        {100, 150, 200},
        { 50,  75, 125},
        {  0,   0, 255}
    };
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            img1.setValue(i, j, values[i][j]);
            img2.setValue(i, j, 10);
        }
    }

    ImageMatrix<int> result = img1.add(img2);

    EXPECT_EQ(result.getValue(0,0), 110);
    EXPECT_EQ(result.getValue(2,2), 265);
}

// Test multiply
TEST(ImageMatrixTest, MultiplyWithIdentity) {
    ImageMatrix<int> img(3,3), identity(3,3);
    int values[3][3] = {
        {100, 150, 200},
        { 50,  75, 125},
        {  0,   0, 255}
    };
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            img.setValue(i, j, values[i][j]);
            identity.setValue(i, j, (i==j ? 1 : 0));
        }
    }

    ImageMatrix<int> result = img.multiply(identity);

    EXPECT_EQ(result.getValue(0,0), 100);
    EXPECT_EQ(result.getValue(1,2), 125);
    EXPECT_EQ(result.getValue(2,2), 255);
}

// Test rowAverage
TEST(ImageMatrixTest, RowAverageCorrect) {
    ImageMatrix<int> img(3,3);
```

```
    int values[3][3] = {
        {100, 150, 200},
        { 50,  75, 125},
        {  0,   0, 255}
    };
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            img.setValue(i, j, values[i][j]);
        }
    }

    EXPECT_DOUBLE_EQ(img.rowAverage(0), (100+150+200)/3.0);
    EXPECT_DOUBLE_EQ(img.rowAverage(2), (0+0+255)/3.0);
}

// Test printImage
TEST(ImageMatrixTest, PrintImageOutputsCorrectly) {
    ImageMatrix<int> img(3,3);
    int values[3][3] = {
        {100, 150, 200},
        { 50,  75, 125},
        {  0,   0, 255}
    };
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            img.setValue(i, j, values[i][j]);
        }
    }
    img.printImage();
    // just check if the matrix printed on the terminal is valid.
}
```

Task 2: Warehouse Inventory Tracker

Problem Statement

A company tracks inventory counts across quarters, product categories, and items.

Implement a template class InventoryTracker<T> that stores counts in a 3D dynamic array addressed as:

stock[quarter][category][item]

- Each quarter has multiple categories.
- Each category contains multiple items.
- Use raw pointers (T***) with new[] / delete[].
- No STL containers for core storage.

Functional Requirements

Constructor / Destructor

- InventoryTracker(int quarters, int categories, int items)
  - Allocate T***, initialize all entries to T{}.
- Destructor frees all memory correctly.

setCount(q, cat, item, value)

Set the count for an item.

getCount(q, cat, item)

Get the count.

itemTotalAcrossQuarters(itemId)

Return the total count of a single item across all quarters and all categories (sum over q, cat at fixed itemId).

categoryRestockList(catId, T threshold)

- Return a dynamically allocated 1D array of length items where:
  - out[item] = 1 if total across quarters for (catId, item) is below threshold, else 0.
  - Caller must delete[] the returned array.

transferQuarter(qFrom, qTo, catId, itemId, T amount)

- If possible, move stock (decrease at (qFrom,catId,itemId), increase at (qTo,catId,itemId)).
- If insufficient stock at source, do nothing (or clamp to zero state behavior).

printSnapshot()

- Print a readable table:
- Per quarter: category totals.
- Per category: list low-stock items (< threshold -pass a param or store one in the class).

GTEST for Warehouse Inventory:

```
// Test template instantiation for int
TEST(InventoryTrackerTemplateTest, InstantiateWithInt) {
    InventoryTracker<int> tracker(2, 2, 2);
    tracker.setCount(0, 0, 0, 10);
    EXPECT_EQ(tracker.getCount(0, 0, 0), 10);
}

// Test template instantiation for double
TEST(InventoryTrackerTemplateTest, InstantiateWithDouble) {
    InventoryTracker<double> tracker(2, 2, 2);
    tracker.setCount(1, 1, 1, 15.5);
    EXPECT_DOUBLE_EQ(tracker.getCount(1, 1, 1), 15.5);
}

// Test template instantiation for char
TEST(InventoryTrackerTemplateTest, InstantiateWithChar) {
    InventoryTracker<char> tracker(1, 1, 2);
    tracker.setCount(0, 0, 0, 'A');
    EXPECT_EQ(tracker.getCount(0, 0, 0), 'A');
}

// --- Core Functional Tests ---

// Test setCount / getCount correctness
TEST(InventoryTrackerFunctionalTest, SetAndGetCount) {
    InventoryTracker<int> tracker(2, 2, 3);
    tracker.setCount(0, 1, 2, 42);
    EXPECT_EQ(tracker.getCount(0, 1, 2), 42);
}

// Test summing totals across quarters
TEST(InventoryTrackerFunctionalTest, ItemTotalAcrossQuarters) {
    InventoryTracker<int> tracker(2, 2, 2);
    tracker.setCount(0, 0, 1, 10);
    tracker.setCount(1, 1, 1, 20);
    EXPECT_EQ(tracker.itemTotalAcrossQuarters(1), 30);
```

```
}

// Test category restock list
TEST(InventoryTrackerFunctionalTest, CategoryRestockList) {
    InventoryTracker<int> tracker(2, 2, 3);
    tracker.setCount(0, 0, 0, 1);
    tracker.setCount(1, 0, 0, 2);
    tracker.setCount(0, 0, 1, 10);
    tracker.setCount(1, 0, 1, 15);

    int* restockList = tracker.categoryRestockList(0, 10);
    EXPECT_EQ(restockList[0], 1);  // below threshold
    EXPECT_EQ(restockList[1], 0);  // above threshold
    delete[] restockList;
}

// Test stock transfer success
TEST(InventoryTrackerFunctionalTest, TransferQuarterSuccess) {
    InventoryTracker<int> tracker(2, 1, 1);
    tracker.setCount(0, 0, 0, 50);
    tracker.transferQuarter(0, 1, 0, 0, 20);
    EXPECT_EQ(tracker.getCount(0, 0, 0), 30);
    EXPECT_EQ(tracker.getCount(1, 0, 0), 20);
}

// Test stock transfer insufficient
TEST(InventoryTrackerFunctionalTest, TransferQuarterInsufficient) {
    InventoryTracker<int> tracker(2, 1, 1);
    tracker.setCount(0, 0, 0, 5);
    tracker.transferQuarter(0, 1, 0, 0, 10);  // too much
    EXPECT_EQ(tracker.getCount(0, 0, 0), 5);  // unchanged
    EXPECT_EQ(tracker.getCount(1, 0, 0), 0);
}

// --- Memory Safety & Edge Cases ---

// Test default initialization
TEST(InventoryTrackerMemoryTest, DefaultInitializedToZero) {
    InventoryTracker<int> tracker(2, 2, 2);
    EXPECT_EQ(tracker.getCount(1, 1, 1), 0);
}

// Test deletion of restock list (avoid memory leaks)
TEST(InventoryTrackerMemoryTest, RestockListMemoryManagement) {
    InventoryTracker<int> tracker(1, 1, 2);
```

```
    int* restockList = tracker.categoryRestockList(0, 5);
    ASSERT_NE(restockList, nullptr);
    delete[] restockList;  // should not crash
}
```